

Lossless Filing JSON

A Deterministic-First Architecture for Financial PDF Extraction with Built-In Consistency Gates

Version 1.0 — **September 2026** **Authors:** The qscreen-filing-tool maintainers **Repository:** <https://github.com/Mine-FNL/qstocks-filing-tool> **License:** MIT (text and figures); sample data per its original license **Cite as:** qscreen-filing-tool maintainers. *Lossless Filing JSON: A Deterministic-First Architecture for Financial PDF Extraction with Built-In Consistency Gates*. v1.0. September 2026.

Abstract

A jurisdiction-agnostic engine that turns any exchange’s annual or interim report into a schema-stable, audit-traceable JSON object — in approximately three seconds per filing, with no API cost, and with a math-identity gate that refuses to ship self-contradictory records. The architecture rests on three bets that distinguish it from existing extraction pipelines: (1) the numbers are read deterministically from the PDF’s tables in code, and the language model only fills *gaps* (audit opinion, notes, segment labels) — so a 270-million-parameter local model produces the same lossless contract as a frontier cloud model; (2) the JSON cannot ship if `Assets ≠ Liabilities + Equity` within a sector-aware tolerance band, with eleven cross-cutting and twenty-five issuer-specific pre-flag rules run before `save()` ; and (3) re-ingesting the same PDF on any engine commit produces bit-identical JSON, attestable back to the engine commit via Sigstore keyless signing and SLSA Build L3 provenance. Across a 124-check regression bench against the current Qatar-listed universe, the engine reaches 100/124 (80.6 %); the bench fails any pull request that drops below the floor. The supply chain is hardened end-to-end: SBOM (CycloneDX 1.5) per release, Sigstore keyless signing, SLSA L3 attestation, CodeQL on every PR and main, `pip-audit` on every PR, and a container image on `ghcr.io` . The pipeline runs offline when local runtimes are available, and the same wheel installs from GitHub Releases, from a PEP 503 simple index on GitHub Pages, or from the container registry — no PyPI Trusted Publisher click required. This paper documents the architecture, the bench, the threat model, and the honest limitations.

1. Introduction

The final-mile problem in financial-document data is well known to anyone who has tried to ship a downstream pipeline that consumes exchange filings. Most extraction tools stop at “give me the text” or “give me a Markdown version of the PDF”. A smaller set returns structured line items. Almost none

of them ship a record whose *internal consistency* is guaranteed, whose *fingerprint* is stable across engine upgrades, and whose *provenance* is cryptographically attestable back to the engine commit that produced it.

The engine described in this paper is the result of six months of work on exactly those three properties. The repository, `qstocks-filing-tool`, contains a jurisdiction-agnostic Python engine plus a Qatar profile that ships by default; other markets (AE, SA, KW) are documented as one-directory-drop extensions. The engine is released under the MIT licence, is published with full provenance and signing infrastructure, and has been bench-marked across 124 cross-filing checks against the current Qatar-listed universe (55 tickers, full-year and interim periods, IFRS and IFRS-for-SME flavours).

This paper is intended for three audiences: (a) engineers building quantitative data pipelines who need a deterministic-ingestion primitive, (b) compliance and audit teams who need cryptographically attestable provenance for filing-data records, and (c) open-source maintainers evaluating supply-chain hardening patterns.

2. Problem

Three failure modes recur in financial-document extraction at scale.

Failure mode 1 — non-deterministic JSON. Re-extracting the same PDF on a different engine commit (a model upgrade, a dependency upgrade, an OCR configuration change) returns a JSON object whose text differs from the previous extraction. This makes downstream pipelines fragile: re-running an ingestion job can move numbers, which makes data diffing meaningless, which in turn makes regulatory reconciliation work unbounded. Existing remedies — post-hoc hashing, manual reconciliation, “frozen extracts” — push the problem downstream rather than solving it.

Failure mode 2 — silently-shipped internal contradictions. Most extraction pipelines ship the JSON regardless of whether the numbers internally agree. A balance sheet whose `Assets ≠ Liabilities + Equity` within a $\pm 5\%$ band will be ingested without warning. A segment whose segment-level revenue does not sum to the consolidated revenue will be ingested without warning. A prior-year comparative column whose currency or scaling does not match the current-year column will be ingested without warning. Downstream pipelines then have to detect and reject these records, which they typically do imperfectly — usually at the dashboard layer, where the failure is most expensive.

Failure mode 3 — opaque provenance. Once a record is in the database, the question “*which engine version produced this row, and what was the PDF source?*” is usually answerable only by reading a database comment or asking the operator. Cryptographic provenance — a signed attestation that binds a JSON record to a specific PDF and a specific engine commit — is uncommon. The result is that *what* the data is, is detached from *how* it was produced.

The engine described in this paper addresses all three.

3. Approach: Three Architectural Bets

3.1 Deterministic-first extraction

The first bet is that **the numbers never pass through the language model**. PDF tables are read in code (`pdfplumber` , with sector-aware table-recovery fallbacks); the model is asked only to fill *gaps* — audit opinion classification, note-text summarisation, segment labels. This is the reason the engine can run on a 270-million-parameter local model with the same numerical contract as a frontier cloud model: numbers are deterministic by construction, and only the gaps are non-deterministic.

The mechanism is the `basis` field on every line item in the output JSON. Each numeric value carries one of two bases: `"parsed"` (read from the PDF's tables by `pdfplumber` , then sign- and scale-corrected by the profile's account-code map), or `"llm"` (only used for non-numeric gaps). A downstream consumer can filter on `"basis": "parsed"` if they want a fully deterministic record, regardless of which model produced the gap fills.

The default profile ships 55 Qatar tickers; adding a new jurisdiction requires only a directory under `profiles/` that exposes a `build_profile(ticker)` function, an account-code map, and a sector taxonomy. The engine itself is jurisdiction-agnostic.

3.2 Math-identity gate

The second bet is that **a record should not ship if it contradicts itself**. The `qscreen_gates` engine runs eleven cross-cutting and twenty-five issuer-specific pre-flag rules before `save()` :

- **Cross-cutting rules** (sector-aware tolerances):
 - `Assets = Liabilities + Equity` ($\pm 2\%$ default; sector overrides for insurance, banks, holding companies)
 - `Revenue + Other Income  $\approx$  Total Operating Income` ($\pm 2\%$)
 - `Net Income  $\rightarrow$  Retained Earnings` reconciliation ($\pm 2\%$)
 - `Cash flow closing balance  $\approx$  Balance Sheet cash` ($\pm 2\%$)
 - Prior-year comparative column currency consistency
 - Negative-number presentation consistency (`-1,234` vs `(1,234)`)
 - Scale consistency (millions vs thousands vs absolute)
 - Auditor-name consistency across periods
 - Reporting-period-end-date consistency
 - Fiscal-year-end-date consistency
 - Jurisdiction-code consistency
- **Issuer-specific pre-flag rules** (a sample, full list in `profiles/qatar/preflag.py`):
 - UDCD: Intangible-property balance must not exceed 47 % of total assets (industry ceiling; historical warning band).
 - ZHCD: Auditor history must include at least one Big-Four predecessor in the prior five years.

- QIGD: Three or more entity-renames in the prior ten years trigger a corporate-history narrative check.
- (... 22 more, documented in `profiles/qatar/preflag.py`)

A record that fails any rule is saved to disk with a `gate_status: "blocked"` field and a `gate_evidence` payload that points at the offending line item; the record is **not** uploaded to `qscreen.app` . The CLI exits non-zero on the gate block so CI catches it; the web app shows a red panel with the evidence; the batch orchestrator skips the upload and surfaces the blocked record in a sidecar report.

The gate runs *before* the JSON is written. This is the key architectural choice: the gate is not a post-hoc validator, it is part of the write path.

3.3 SHA-256 cross-filing fingerprints

The third bet is that **the JSON's hash is part of its identity**. The engine computes a canonical SHA-256 over the normalised JSON content and embeds it in the record as `fingerprint` . Re-ingesting the same PDF on any engine commit produces, after canonicalisation, the same hash. The fingerprint excludes engine-version-specific fields (`@engine` , `@extracted_at`) so the contract is stable across upgrades.

Combine this with Sigstore keyless signing on every release:

```
gh release download v1.6.0 --pattern '*.sig'
gh attestation verify qscreen_filing_tool-1.6.0-py3-none-any.whl \
  --bundle qscreen_filing_tool-1.6.0-py3-none-any.whl.sig
# → verified against Fulcio + Rekor; SLSA v1 receipt attached
```

...and you have a cryptographic chain:

```
PDF (sha256:X) → engine commit (git+sigstore) → JSON (sha256:Y) → fingerprint match:
```

This is the property that makes audit trails work *without trusting the operator*: the question “did this PDF produce this JSON on this commit?” is answerable from the public attestations alone.

4. Implementation

The engine is a Python package; the CLI is `qscreen-ingest` ; the web app is `qscreen-app` . Both are entry points of the same wheel.

4.1 Profile pluggability

A profile directory under `profiles/` exposes three things:

1. A `JURISDICTION_NAME` constant (e.g., `Qatar` , `UAE`).

2. A `build_profile(ticker: str) -> Profile` factory function that returns a `Profile` object — sector, fiscal calendar, currency, framework, account-code map, audit-firm history, and pre-flag rules.
3. A sector taxonomy (banks, insurance, industrial, real-estate, holding).

The CLI defaults to `--jurisdiction qatar` if `--symbol` is set. If `--symbol` is omitted, no profile is loaded and the engine works purely on what's in the PDF — useful for one-off foreign issuers.

The `qatar` profile ships with 55 tickers, full pre-flag rules, and the 11 cross-cutting gate rules. The engine has been run end-to-end against the Qatar-listed universe; the bench (Section 5) is the regression floor.

4.2 Supply chain hardening

Every release goes through:

1. **Build** — `python -m build` produces wheel + sdist.
2. **SBOM** — `cyclonedx-py` generates CycloneDX 1.5 SBOM in both JSON and XML, attached to the GitHub release.
3. **Sigstore keyless signing** — `sigstore-python` signs the wheel against Fulcio (OIDC) + Rekor (transparency log). The signature bundle is attached to the release.
4. **SLSA Build L3 attestation** — `actions/attest-build-provenance@v1` generates the SLSA v1 provenance receipt. Verifiable with `gh attestation verify`.
5. **Container** — `docker buildx build` produces a multi-arch (`linux/amd64` , `linux/arm64`) image on `ghcr.io/Mine-FNL/qstocks-filing-tool:vX.Y.Z`.

The PyPI Trusted Publisher workflow is staged but dormant (a maintainer-only UI click on `pypi.org` is required to enable). Until then, three public install paths exist:

- `pip install https://github.com/Mine-FNL/qstocks-filing-tool/releases/latest/download/<wheel>`
- `pip install --extra-index-url https://mine-fnl.github.io/qstocks-filing-tool/simple/ qscreen-filing-tool`
- `docker pull ghcr.io/Mine-FNL/qstocks-filing-tool:vX.Y.Z`

All three install the same wheel that `.github/workflows/release.yml` builds, gates on the bench regression check, and uploads to GitHub Releases on every `v*` tag push.

5. Bench Results

The bench (`qscreen_eval.py --json`) runs 124 cross-filing checks against the current Qatar-listed universe. Each check is a tuple of (`filing`, `gate`, `expected_value`); a check passes if the engine produces the expected output for the given filing.

Metric	v1.0	v1.6.0
Cross-filing checks	124	124
Passing checks	87	100
Pass rate	70.2 %	80.6 %
Pre-flag rule coverage	9 / 25	25 / 25
Bench wall-time (CI, ubuntu)	9 min	11 min

The regression floor is $100/124 = 80.6\%$. Any pull request that drops below the floor fails the `bench.yml` gate and is blocked from merge.

The full per-filing breakdown is published at <https://mine-fnl.github.io/qstocks-filing-tool/demo.html>, regenerated on every release tag from `qscreen_eval.py --json` output.

6. Threat Model

Threat	Mitigation
Compromised dependency injects code at install time	SBOM per release; Sigstore keyless signature; SLSA L3 provenance; <code>pip-audit</code> on every PR.
Engine commit produces wrong extraction	Gate runs before <code>save()</code> ; non-zero exit on block; CI bench catches regression at PR time.
LLM provider returns subtly-wrong values	Numbers never pass through the LLM (<code>basis: "parsed"</code>); only gap-fills are LLM-sourced.
Attacker swaps PDF for a different filing	SHA-256 of the PDF is captured at extraction; recorded alongside the JSON fingerprint.
Audit trail tampering	Sigstore-signed attestations are public; verification does not require operator trust.
Run-time prompt injection via the PDF itself	Numbers are parsed from tables, not extracted from prompt text; prompts are read-only.

7. Limitations

The engine has honest ceilings. We document them here rather than gloss over them.

- **OCR of image-only PDFs is opt-in.** The default extractor requires a text layer; scanned pages need `--ocr auto` plus the `ocr` extra and the `tesseract` system binary.

- **Tier-B ceilings on harder benchmarks.** BigCodeBench is at 28.4 % (vs. ~50 % for frontier models) — the engine’s deterministic-first architecture caps the upper-bound on hard multi-step reasoning tasks.
- **Single-language coverage.** The default profile is English (QSE publishes English-language filings). Arabic / Urdu / Malayalam filings need a language-specific profile drop.
- **The gate is not a substitute for human review.** The math-identity gate catches self-contradictions but does not catch misclassifications (e.g., an audit opinion labeled “unqualified” when the auditor’s emphasis-of-matter paragraph contradicts it).
- **The 270M local model is not a general-purpose LLM.** It is good at the specific asks the engine makes (audit-opinion classification, short-label extraction). It is not a chatbot.

8. Roadmap

Now → v1.7.0 (Q4 2026)

- Second jurisdiction (UAE) profile drop
- Whisper-generated captions for the WHY-videos (multi-language)
- 9:16 vertical cuts for LinkedIn / X / TikTok

v2.0 (Q1 2027)

- Multi-PDF batch with SQLite-backed resume (the orchestrator already supports it via `qscreen-ingest --batch`; the resume ledger is a pre-flag-rule addition)
- Audit-opinion classification model trained on the gate-flagged disagreements (active-learning loop)
- PyPI Trusted Publisher activation (gated on maintainer UI click)

Future

- Tier-mixing validation against harder benchmarks (BigCodeBench + LiveCodeBench)
- Financial-document LLM benchmark suite contribution
- OpenSSF Scorecard “gold” tier

9. Conclusion

Financial-document extraction is a special case of “give me a structured record I can ingest.” The thing that makes it special is that the *integrity* of the record matters as much as its *content*: a number that is internally contradicted is worse than a missing number, because downstream pipelines will silently trust it.

The three architectural bets in this paper — deterministic-first extraction, math-identity gate, and SHA-256 cross-filing fingerprints — are the minimum sufficient set of properties we found to make financial filing extraction trustworthy at scale. The implementation is open-source (MIT), the bench is

public (regenerated on every release), the supply chain is hardened end-to-end (SBOM + Sigstore + SLSA L3), and three public install paths exist that do not require a maintainer-only UI click.

We invite contributors, sponsors, and grant programs to engage with the work. The architecture is reproducible, the bench is the gate, and the bench is honest.

References

- [1] CycloneDX 1.5 specification. <https://cyclonedx.org/specification/overview/>
- [2] Sigstore project. <https://www.sigstore.dev/>
- [3] SLSA Build L3 specification. <https://slsa.dev>
- [4] pdfplumber. <https://github.com/jsvine/pdfplumber>
- [5] OpenSSF Scorecard. <https://scorecard.dev/>
- [6] PEP 503 — Simple Repository API. <https://peps.python.org/pep-0503/>
- [7] GitHub Actions OIDC + Trusted Publishers. <https://docs.github.com/en/actions/security-guides/automatic-token-authentication>
- [8] Qatar Stock Exchange — listed-companies disclosure portal. <https://www.qse.com.qa/>
- [9] International Financial Reporting Standards (IFRS). <https://www.ifrs.org/>
- [10] AAOIFI Financial Accounting Standards. <https://aaoifi.com/>
- [11] Trino / Presto SQL-on-files benchmark methodology. <https://trino.io/>
- [12] HashiCorp Vault — dynamic-secrets for CI. <https://www.vaultproject.io/>

Appendix A — Reproducing the bench

```
git clone https://github.com/Mine-FNL/qstocks-filing-tool
cd qstocks-filing-tool
pip install -e ".[dev]"
python qscreen_ingest.py --self-test           # 499-test contract gate
python qscreen_eval.py --json > bench.json    # 124-check regression bench
python docs/build_demo.py bench.json         # → demo.html
```

Appendix B — Verifying a release

```
VERSION=v1.6.0
gh release download ${VERSION} --pattern '*.whl' --pattern '*.sig' \
  --pattern '*.sbom.*' --pattern '*.intoto.jsonl'
gh attestation verify qscreen_filing_tool-${VERSION#v}-py3-none-any.whl \
  --bundle qscreen_filing_tool-${VERSION#v}-py3-none-any.whl.sig
# → verified against Fulcio + Rekor; SLSA v1 receipt attached
```

Appendix C — Citing this paper

```
@techreport{qscreen2026whitepaper,
  title      = {Lossless Filing JSON: A Deterministic-First Architecture
                for Financial PDF Extraction with Built-In Consistency Gates},
  author     = {{qscreen-filing-tool maintainers}},
  year      = {2026},
  month     = sep,
  number    = {v1.0},
  institution = {Mine-FNL},
  url       = {https://github.com/Mine-FNL/qstocks-filing-tool},
  note      = {MIT licence}
}
```